

PRODUCT TOUR

Qualityfolio

Quality You Can Prove.

Test Management as Code (TMaC) — test artifacts authored in Markdown, versioned in Git, executed via CLI or CI, and stored as immutable, queryable SQL evidence in surveilr.

Today's Tour

01

Test Debt Is Audit Debt

Why traditional QA fails compliance

02

TMaC Architecture

Markdown + Git + SQL = provable quality

03

The Ecosystem

Qualityfolio + Opsfolio Suite + Fleetfolio

04

Feature Tour

Strategy, cycles, evidence, traceability

05

SQL Evidence

Every test is a query. Every result is evidence.

06

Deploy & Close

Integrations, deployment, and next steps

Test Debt Is Audit Debt

Organizations that pass functional tests still fail audits — because they cannot prove what happened.

Why QA Evidence Fails Audits

Test Cases in Spreadsheets

Excel and TestRail data is non-versioned, manually edited, and changes without trace. No Git history.

Evidence Assembled After the Fact

Auditors arrive and teams scramble to screenshot and export. Evidence is assembled, not generated.

No Requirement Traceability

Cannot prove which test covers which control. Frameworks demand traceability; TMS tools rarely deliver it.

Flaky Tests Silently Pass

A test that passes 80% of the time looks compliant at audit time but is a governance risk every day.

The Result

Test-ready. Audit-blind.

Tests pass in CI. Evidence vanishes. Auditors arrive to find screenshots taken yesterday, requirements traced manually to spreadsheets, and no provable record of what the system actually did.

TMaC: Test Management as Code

Traditional TMS: evidence is collected after the fact. Qualityfolio: evidence is the byproduct of execution.

Traditional TMS (TestRail, Xray, qTest)

- Test cases stored in proprietary DB — no Git, no version history
- Evidence uploaded manually by QA engineers after test runs
- Requirement links maintained by hand — break silently on change
- Compliance reporting requires dedicated export/assembly cycles
- AI cannot read or generate test cases in proprietary formats
- Vendor lock-in — evidence inaccessible when contract ends
- Offline or air-gapped deployment usually unsupported

Qualityfolio (TMaC)

- Markdown + YAML authoring — plain text, Git-versioned with full PR review workflow
- Evidence is the execution byproduct — captured automatically on every run
- FII (Foreign Integration Identifiers) link cases to controls at authoring time
- SQL query against surveilr generates compliance package in minutes
- AI can read, generate, and review Markdown test cases natively
- Open standards — Markdown + SQLite; evidence survives any vendor change
- Full offline / air-gapped support for FDA-regulated lab environments

Qualityfolio in the Opsfolio Ecosystem

Test evidence flows directly into Opsfolio Suite's Evidence Warehouse, where it auto-maps to compliance controls.

Qualityfolio

Test Management as Code (TMaC)

- Strategies, suites, plans, cycles authored in Markdown + Git
- Automated + manual execution with full provenance capture
- Evidence stored as immutable SQL in surveilr
- FII links test cases to SOC 2, HIPAA, FDA 21 CFR, ISO controls
- AI-assisted test case generation from requirements
- Offline / air-gapped support for validated lab environments

Opsfolio Suite

GRC Engineering Platform

Evidence Warehouse

Test execution records auto-mapped to compliance controls

Quality Intelligence

Pass/fail trends and coverage gaps surface as compliance signals

Audit Center

Qualityfolio evidence populates SOC 2, HIPAA, FDA audit packages

Controls Library

Test results validate executable control predicates in real time

Risk & Threat

Flaky tests and test failures surface as operational risk signals

Fleetfolio

SQL-Native EDR

- Endpoint posture feeds Qualityfolio asset context
- Fleet TEM findings trigger security test cycles
- Shared surveilr DB — one SQL schema for both
- Fleet evidence + test evidence in one audit package

fleetfolio.dev

Operational Truth™ in Testing

Quality is only provable when declared intent (Expected Tests) and runtime reality (Observed Results) are continuously aligned.

COMPLIANT

Tests + Evidence Align

Operational Truth™ achieved

Test cases executed with captured results.
Cryptographic provenance. Auditor-defensible. The control is proven, not claimed.

GAP SIGNAL

Tests Lack Evidence

Assurance is incomplete

Test cases exist but no execution evidence was captured. Intent without proof. Surfaces as an OT™ gap signal — never silently assumed compliant.

GOVERNANCE FAIL

Evidence Without Tests

Governance is broken

Execution records exist with no corresponding test case. Untrusted evidence. A compliance liability — cannot be traced to approved intent.

In Opsfolio Suite: GAP and GOVERNANCE FAIL states surface as Operational Truth™ gap signals — uninstrumented controls are never silently assumed compliant.

Test Strategy & Suite Management

Define what must be tested — in Git, not spreadsheets.

- Strategies authored in Markdown with YAML metadata — version-controlled, PR-reviewed
- Suite hierarchy: Strategy → Suite → Plan → Cycle → Case → Evidence
- Requirements attached at plan level via FII — one requirement governs many cases
- Status fields (Active / Deprecated / Draft) tracked in YAML, queryable in SQL
- Multi-team support: separate projects share strategy artifacts via Git references

Git

Version Control

YAML

Machine-Readable

SQL

Queryable Hierarchy

https://demo.qualityfolio.dev/suites

Qualityfolio

Home / Test suite

Test suite is a collection of test cases designed to verify the functionality, performance, and security of a software application. It ensures that the application meets the specified requirements by executing predefined tests across various scenarios, identifying defects, and validating that the system works as intended..

Search...

SUITE NAME ↕	TITLE ↕	TEST CASES ↕	CREATED DATE ↕	CREATED BY ↕
SUITE-REQ-SUR-001	Qualityfolio Comprehensive Suite	10	04-13-2026	QA Team

© 2026 Qualityfolio. Test assurance as living Markdown.

Test Cycle Execution

Every run is a timestamped, immutable evidence record.

- Cycles are timeboxed execution batches — each cycle produces a distinct evidence set
- Manual, automated (CLI), and CI/CD execution paths — same evidence schema for all
- Each case result stored as JSON with start/end timestamps and step-level details
- Cycle approval workflow: created_by + approved_by captured in YAML at execution time
- Re-run history preserved — every cycle's results are independently queryable in SQL

TCY

Cycle IDs

Auto

CI/CD Native

Hash

Proof of State

[Home](#) / [Test Cycle History](#)

Provides a cycle-wise summary of test execution, showing total test cases and status-wise counts (passed, failed), with optional date-range filtering to analyze results by cycle creation and ingestion time.

From date

03/02/2026



To date

05/31/2026



Search...

CYCLE ↕	CYCLEDATE ↕	TOTAL TEST CASES ↕	TOTAL PASSED ↕	TOTAL FAILED ↕
1.0.2	04-13-2026	3	1	1
1.0.3	04-13-2026	3	3	0
1.0.4	04-13-2026	4	4	0

© 2026 Qualityfolio. Test assurance as living Markdown.

Test Case Authoring

Plain Markdown. Git PR workflow. AI-assisted generation.

- Cases authored as TC-XXXX.case.md with YAML: priority, type, scenario_type
- Body sections: Description, Pre-Conditions, Test Steps, Expected Result
- Compliance metadata inline: requirement_ids, regulatory_context, risk_level
- AI generates validation protocols from requirements: “Generate FDA cases from this URS”
- Cases reusable across cycles — reference by case ID; no duplication or drift

MD

Open Format

AI

Case Generation

FII

Control Links

The screenshot displays the Qualityfolio Markdown Editor interface. The browser address bar shows `https://demo.qualityfolio.dev/cases`. The interface includes a sidebar with a 'MAIN MENU' containing 'Generator', 'Markdown Editor' (selected), 'View Dashboard', and 'Settings'. The main area is titled 'Markdown Editor' and 'Edit and manage markdown files'. It shows an 'Explorer' on the left with a file 'example-artifact.md*'. The main editor displays 'example-artifact.md (Unsaved changes)' with tabs for 'PREVIEW', 'HIERARCHY', 'MARKDOWN', and 'JSON'. Below the tabs are controls for 'SELECT ALL', '0 SELECTED', and various filters for 'SCENARIO TYPE', 'EXECUTION TYPE', 'TAGS', 'ASSIGNEE', and 'RUN CYCLE'. The main content area shows a 'Qualityfolio' plan with a 'Suite: Qualityfolio Comprehensive Suite' containing three test cases:

- ☐ Verify successful login using registered email and correct password (ID: TC-QUALITYFOLIO-001, Priority: Medium, Assignee: John Doe)
- ☐ Verify login failure due to network timeout despite correct credentials (ID: TC-QUALITYFOLIO-002, Priority: Medium, Assignee: Sarah Kim)
- ☐ Verify secure transport (HTTPS) is enforced on the login page (ID: TC-QUALITYFOLIO-006, Priority: Medium, Assignee: John Doe)

At the bottom, a footer reads: © 2026 Qualityfolio. Test assurance as living Markdown.

Evidence Capture & Dashboard

Live pass/fail/bug metrics. No manual evidence assembly.

- Results auto-ingest via surveilr on every CI/CD run or manual execution
- Dashboard: total cases executed, pass/fail rates, active bugs, execution trends
- Evidence hash: SHA-256 + Git commit SHA + executor identity = cryptographic provenance
- Bug evidence declared at point of failure — BUG-XXXX.md linked to the failing run
- Dashboard auto-updates on ingest — auditors see live coverage state, not last export

Live

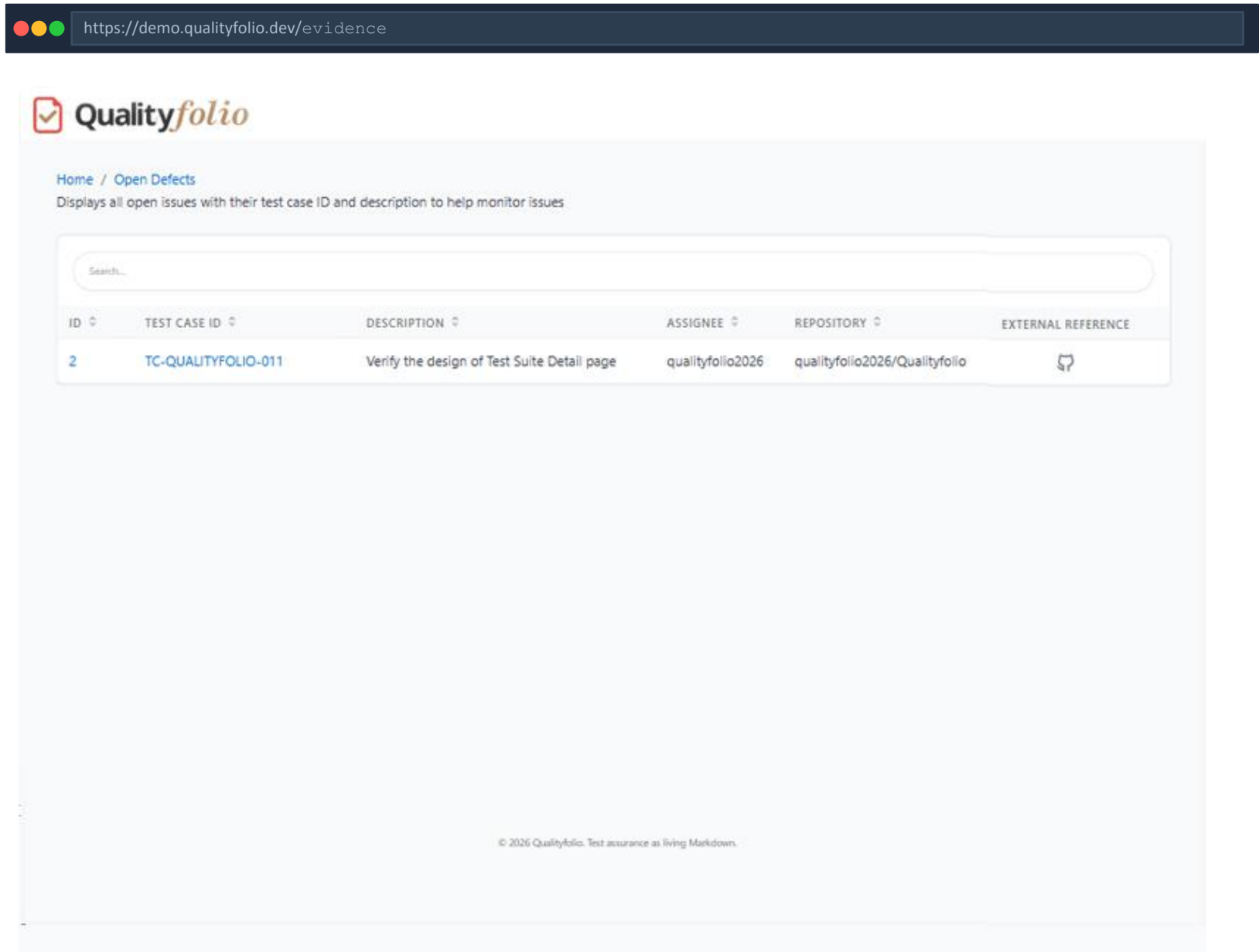
Dashboard

SHA

Evidence Hash

Zero

Manual Steps



Requirement Traceability

Every test mapped to a control. Every control provably covered.

- FI (Foreign Integration Identifiers) link cases to framework controls at authoring time
- One test case satisfies multiple frameworks: SOC 2 + HIPAA + ISO in one record
- Coverage matrix queryable via SQL: which controls have passing evidence right now?
- Gap detection: controls with no executed test cases surface as coverage gaps in real time
- Traceability flows to Opsfolio Suite — evidence auto-tagged at ingest; no re-mapping

N:M

Case:Control Map

SQL

Coverage Query

Live

Gap Detection

The screenshot shows a web browser window with the URL `https://demo.qualityfolio.dev/traceability`. The page features the Qualityfolio logo and a title "REQUIREMENT TRACEABILITY". Below the title is a search bar with the placeholder text "Search...". A table displays the following data:

REQUIREMENT ID ↕	TOTAL TEST CASES ↕	TOTAL PASSED ↕	TOTAL FAILED ↕	TOTAL CLOSED ↕
REQ-QUALITYFOLIO-01	10	8	1	1

Below the table, it indicates "(Page 1 of 1)". At the bottom of the page, there is a copyright notice: "© 2026 Qualityfolio. Test assurance as living Markdown."

Bug Tracking & Defect Lifecycle

Defects as code. Linked to evidence. Never orphaned.

- Bugs declared at point of failure as BUG-XXXX.md — directly linked to the failing run
- YAML metadata: severity, priority, assigned, status — all queryable in SQL
- Lifecycle states: OPEN → IN PROGRESS → RESOLVED → REJECTED — tracked in Git history
- Flaky test detection: pass-rate queries identify cases with >5% failure variance
- Jira / ServiceNow auto-ticket integration — bug creates ticket at detection time

Git

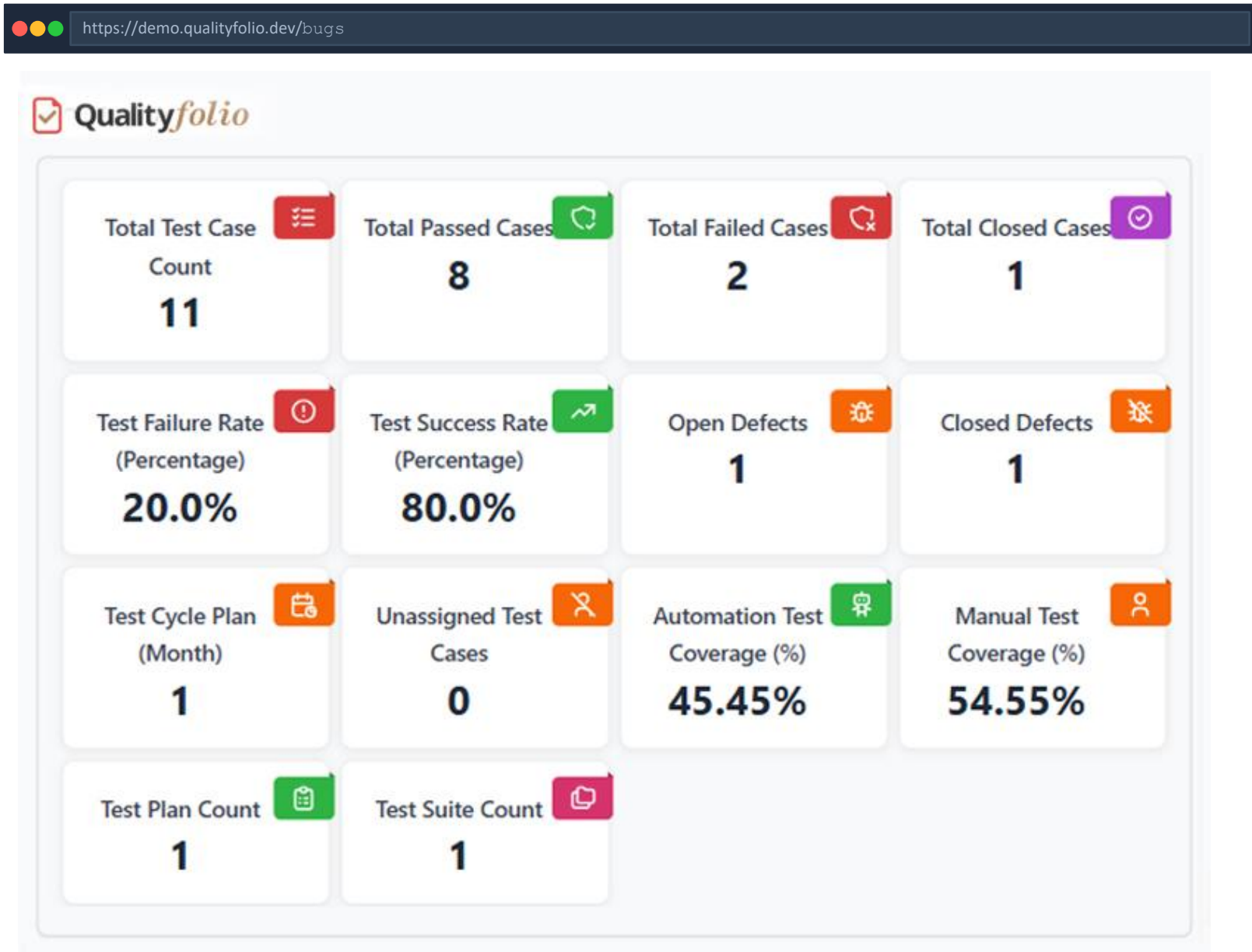
Bug History

SQL

Flaky Detect

Auto

Jira Tickets



Every Test Is a Query. Every Result Is Evidence.

Standard SQL — testable, version-controllable, auditor-defensible.

SOC 2: Access control tests Q3 2025

```
SELECT ose.exec_nature AS test,
       ose.exec_status AS status,
       os.orch_started_at AS run_time
FROM orchestration_session_exec ose
JOIN orchestration_session os
  ON ose.orchestration_session_id
     = os.orchestration_session_id
WHERE ose.exec_nature
      LIKE '%access-control%'
      AND os.orch_started_at
         BETWEEN '2025-07-01'
         AND '2025-09-30'
ORDER BY os.orch_started_at DESC;
```

Maps to: SOC 2 CC6.1 · CC6.2 · CC7.1

FDA: Computer system validation report

```
SELECT
  json_extract(
    ose.diagnostics_json,
    '$.test_id') AS test_id,
  json_extract(
    ose.diagnostics_json,
    '$.validation_category')
    AS category,
  ose.exec_status,
  json_extract(
    ose.diagnostics_json,
    '$.requirement_ids') AS reqs
FROM orchestration_session_exec ose
WHERE json_extract(
  ose.diagnostics_json,
  '$.validation_category')
      = 'computer_system_validation';
```

FDA 21 CFR Part 11 · GxP validation

Flaky tests: assurance risk detection

```
SELECT
  ose.exec_nature AS test,
  COUNT(*) AS runs,
  SUM(CASE WHEN
        ose.exec_status='PASSED'
        THEN 1 ELSE 0 END)
    AS passes,
  ROUND(100.0 * passes
        / COUNT(*), 2)
    AS pass_rate
FROM orchestration_session_exec
GROUP BY ose.exec_nature
HAVING COUNT(*) > 5
      AND pass_rate < 95
ORDER BY pass_rate ASC;
```

Full audit trail — who, when, before/after

Built-In Compliance Mapping

Every test case maps to a control. Audit packages are a SQL query, not a sprint.

SOC 2 Type I & II CC6.1, CC6.2, CC7.1, CC8.1	FDA 21 CFR Part 11 Part 11.10 (a-k), Part 11.50	HIPAA §164.312 (a) (1), §164.312 (c) (1)
ISO 27001 A.9.4.1, A.10.1.1, A.12.6.1	GxP / CSV IQ, OQ, PQ, DQ protocols	CMMC Level 2+ AC.1.001, CM.2.061, SI.1.210

How This Reaches Opsfolio Suite

- Qualityfolio test execution records are ingested into the Opsfolio Suite Evidence Warehouse automatically.
- Each evidence record is tagged with framework controls at ingest time — SOC 2, HIPAA, FDA in one record.
- The Opsfolio Audit Center queries Qualityfolio evidence to populate audit packages on demand. No manual assembly.
- Coverage gaps surface as Operational Truth™ gap signals — uninstrumented controls are never assumed compliant.

Run Where Your Evidence Needs to Live

Evidence sovereignty: your evidence, your infrastructure, your audit chain.

GIT

Git-Hosted / GitOps

Test artifacts live in your own Git repo. CI/CD ingests evidence into surveilr automatically. Audit trail = your commit history.

- Your repo, your CI
- Full commit-level history
- GitHub/GitLab/Bitbucket
- GitOps-compatible

LOCAL

Local / Self-Hosted

Run surveilr on any machine. Evidence stored in local SQLite. No internet required. Auditors access the db directly via SQL.

- No internet required
- SQLite evidence store
- Your infrastructure
- Full SQL access

AIRGAP

Air-Gapped / Validated Lab

Maximum control for FDA-regulated, GxP, or classified environments. Manual transfer. Offline operation. Sync when permitted.

- No external network
- Manual evidence sync
- FDA 21 CFR Part 11
- GxP validated mode

CICD

CI/CD / Cloud-Native

Jenkins, GitHub Actions, GitLab CI. surveilr ingest runs as a pipeline step. Evidence generated automatically on every PR merge.

- Jenkins / GH Actions
- PR-triggered evidence
- Slack / Jira alerts
- Central surveilr DB

Our Guarantee

“We guarantee your audit outcome. If you follow our process and don’t pass, we keep working — at our cost — until you do.”

98%

Audit Pass Rate

Months

To Audit-Ready

SQL

Evidence on Demand

CaaS

Delivery Model

Compliance as a Service (CaaS) Delivery

Netspective deploys Qualityfolio against your test infrastructure, CI/CD pipelines, and compliance framework. Pre-built strategy templates cover 60–70% of SOC 2 and FDA testing surface from day one. We deliver compliance engineering capability — not just a license.

Ready to Prove Your Quality?

Qualityfolio · Opsfolio Suite · Fleetfolio

1

Author Your First Test

Run `qf init --template=soc2` and author your first test case in Markdown. Evidence starts flowing on your next CI run.

2

Execute a Cycle

Execute a test cycle against your staging environment. Every result is a timestamped, immutable evidence record in surveilr.

3

Generate an Audit Package

Within 30 days: query surveilr to produce your first SOC 2 or FDA evidence package. No sprint. No screenshots.

Contact Us

QA: qualityfolio.dev

Suite: opsfolio.com

Fleet: fleetfolio.dev

Email: sales@opsfolio.com

[Schedule a Live Demo](#)